

Machine Learning I: Hands-On 6

Comparing classification performance

The goal of this hands-on is to study how classification algorithms should be compared empirically. The emphasis is not only on obtaining an average score, but also on understanding **variability across resamples** and on distinguishing evaluation within one dataset from comparison across several datasets.

The practical uses three classification tasks, `ds1`, `ds2`, and `ds3`, followed by a second collection of results covering ten tasks.

Consider the following classifiers:

- k-nearest neighbours (k-NN);
- decision tree;
- linear discriminant analysis (LDA);
- Gaussian naive Bayes;
- support vector machine (SVM) with linear kernel;
- SVM with polynomial kernel;
- SVM with radial-basis-function (RBF) kernel.

For k-NN and the SVMs, standardization must be learned **inside each training fold** so that information from the held-out fold is not used during preprocessing.

6.1 Inspect three classification tasks

Load `ds1`, `ds2`, and `ds3`. Each dataset contains two predictors, `X1` and `X2`, and a target variable, `Class`.

1. Inspect the number of observations and predictors in each task.
2. Inspect the number of observations belonging to each class.
3. Before fitting any classifier, consider the following question:
Should one classifier necessarily perform best on all three tasks?
4. Explain why similar dataset sizes and similar class counts do not imply that the tasks have the same learning difficulty.

ML insight: class counts do not describe class geometry

Two datasets can have identical numbers of examples per class and still require very different decision functions. Separation, overlap, boundary shape, local structure, and feature distributions are not visible from class counts alone.

6.2 Evaluate all classifiers using shared cross-validation folds

Use **10-fold stratified cross-validation** to estimate the accuracy of all seven classifiers on each dataset.

Use the **same cross-validation folds for every classifier** within a task.

1. Define a stratified 10-fold procedure with shuffling and a fixed random state.
2. Evaluate every classifier using accuracy.
3. Store the fold-wise results for each dataset in a dataframe in which:
 - each row corresponds to one held-out fold;
 - each column corresponds to one classifier;
 - each cell contains the accuracy obtained on that fold.

The result for each task should therefore contain **10 rows and 7 columns**.

4. Inspect the result table for `ds2`.
5. Explain what one row of that table represents.
6. Explain why standardization must be fitted separately inside every training fold rather than once using the complete dataset.

Prediction before running the experiment: Why is it useful for all classifiers to be evaluated on the same held-out observations?

6.3 Summarise performance and variability

For `ds2`, compute for every classifier:

- mean cross-validation accuracy;
 - sample standard deviation of the ten fold accuracies.
1. Order the classifiers by mean accuracy.
 2. Find two classifiers with similar average performance and compare their standard deviations.
 3. Explain what information is lost when only the mean score is reported.

Prediction before computing the summary: Can two models have similar mean accuracy but different variability?

ML insight: performance and stability are different properties

Mean cross-validation accuracy summarises the centre of the observed scores. Standard deviation describes how much those scores vary across the held-out folds. A model can therefore have a strong average while still being sensitive to the particular observations used for training and evaluation.

6.4 Compare the fold-wise score distributions

Use a boxplot to visualise the ten fold accuracies for the seven classifiers on `ds2`.

Then compare the mean accuracies obtained on `ds1`, `ds2`, and `ds3`.

1. Describe the amount of overlap between the score distributions on `ds2`.
2. Compare the ordering of the classifiers across the three tasks.
3. Identify algorithms whose relative performance changes substantially from one task to another.
4. Explain why a small difference in average accuracy should be interpreted together with the observed variability.

Prediction before plotting: Will the ordering of classifiers by mean accuracy describe what happens on every individual fold?

The fold results are used here to describe estimated performance and variability under the chosen resampling procedure. They should not be treated as ten independent experimental replications because the cross-validation training sets overlap.

6.5 Compare algorithms across ten tasks

The repository contains precomputed 10-fold cross-validation results for ten classification tasks. Load them using the course helper:

```
from mlcourse.labs import load_task_results

task_means = load_task_results()
```

The helper reads the packaged evaluation results and returns one **dataset-by-algorithm performance table**.

For each task, rank the classifiers from best to worst:

- rank 1 corresponds to the highest mean accuracy;
 - ties receive their average rank.
1. Inspect the dataset-by-algorithm mean-accuracy table.
 2. Compute the corresponding rank table.
 3. Identify at least one tied rank.
 4. Identify a change in classifier ordering between two tasks.
 5. Explain why averaging accuracies and averaging ranks need not produce the same ordering.
 6. Explain what changes conceptually when one row of the analysis represents an entire dataset rather than one cross-validation fold.

ML insight: the experimental unit has changed

Within one task, folds describe resampling variation for that task. Across multiple tasks, each dataset contributes one summary result. The question is now whether algorithms tend to behave consistently better or worse across different learning problems.

6.6 Average ranks and the Friedman test

Use the ten dataset-level mean accuracies to compare all seven classifiers across tasks.

1. Apply the Friedman test using `friedmanchisquare()` from `scipy.stats`.
2. Use a significance level of $\alpha = 0.05$.
3. Compute the average rank of each classifier across the ten tasks.
4. Plot the average ranks, where a lower rank is better.
5. Report the classifier with the best observed average rank.
6. Interpret the Friedman p-value.
7. Explain what an omnibus Friedman result does and does not establish.

Prediction before running the test: Does the classifier with the best average rank need to be the best classifier on every task?

Interpretation

A significant Friedman result provides evidence that the algorithms do not all behave equivalently across the collection of tasks, under the assumptions of the test. It does not, by itself, identify which specific pairs of algorithms differ, and it does not imply that one classifier is universally best.